Sample Code

```
import pickle

from django.shortcuts import render

from django.contrib import messages

from sklearn.svm import SVC

from .forms import UserRegistrationForm

from .models import UserRegistrationModel

from django.conf import settings

import os

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,

classification_report

from sklearn.metrics.pairwise import cosine_similarity

from sklearn.preprocessing import LabelEncoder

import matplotlib

matplotlib.use('Agg')

import matplotlib.pyplot as plt

import seaborn as sns

import numpy as np

import io

import base64

# Global variables

tfidf = TfidfVectorizer(stop_words='english', ngram_range=(1, 2), sublinear_tf=True,
```

```
max_features=500)

rf_model = RandomForestClassifier(n_estimators=30, max_depth=12, random_state=42)

label_encoder = LabelEncoder()

def plot_to_base64():

buffer = io.BytesIO()

plt.savefig(buffer, format='png', bbox_inches='tight')

buffer.seek(0)

image_png = buffer.getvalue()

graph = base64.b64encode(image_png).decode('utf-8')

buffer.close()

plt.close()

return graph

def UserRegisterActions(request):

if request.method == 'POST':

form = UserRegistrationForm(request.POST)

if form.is_valid():

print('Data is Valid')

form.save()

messages.success(request, 'You have been successfully registered')

form = UserRegistrationForm()

return render(request, 'UserRegistrations.html', {'form': form})

else:

# Construct a descriptive message based on the actual validation errors

errors = []

for field, field_errors in form.errors.items():

field_name = field.replace('_', ' ').capitalize()
```

```
errors.append(f"{field_name}: {', '.join(field_errors)}")

error_msg = "Registration failed: " + " | ".join(errors)

messages.success(request, error_msg)

print("Invalid form:", form.errors)

else:

form = UserRegistrationForm()

return render(request, 'UserRegistrations.html', {'form': form})

def UserLoginCheck(request):

if request.method == "POST":

loginid = request.POST.get('loginid')

pswd = request.POST.get('pswd')

print("Login ID = ", loginid, ' Password = ', pswd)

try:

check = UserRegistrationModel.objects.get(loginid=loginid, password=pswd)

status = check.status

print('Status is = ', status)

if status == "activated":

request.session['id'] = check.id

request.session['loggeduser'] = check.name

request.session['loginid'] = loginid

request.session['email'] = check.email

print("User id At", check.id, status)

return render(request, 'users/UserHomePage.html', {})

else:

messages.success(request, 'Your Account Not at activated')

return render(request, 'UserLogin.html')
```

```
except Exception as e:
print('Exception is ', str(e))
messages.success(request, 'Invalid Login id and password')
return render(request, 'UserLogin.html', {})
def UserHome(request):
return render(request, 'users/UserHomePage.html', {})
def DatasetView(request):
path = os.path.join(settings.MEDIA_ROOT, 'jobs.csv') # Removed redundant 'media'
df = pd.read_csv(path, nrows=5000)
df_html = df.to_html(classes='table table-striped', index=False)
return render(request, 'users/viewdataset.html', {'data': df_html})
def Training(request):
jobs_df = pd.read_csv('media/jobs.csv', nrows=5000)
jobs_df['combined_features'] = jobs_df['Company'].fillna('') + " " + jobs_df['Required
Skills'].fillna('') + " " + jobs_df['Location'].fillna('') + " " + jobs_df['Salary'].fillna('') + " " +
jobs_df['Description'].fillna('')
# TF-IDF + Label encoding with limits to prevent MemoryError
tfidf.max_features = 500
X = tfidf.fit_transform(jobs_df['combined_features'])
y = label_encoder.fit_transform(jobs_df['Category'])
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=88)
rf_model.n_estimators = 30
rf_model.max_depth = 12
rf_model.random_state = 42
rf_model.fit(X_train, y_train)
```

```
y_pred = rf_model.predict(X_test)

# Evaluation metrics

accuracy = accuracy_score(y_test, y_pred)

precision_rf = precision_score(y_test, y_pred, average='weighted', zero_division=0)

recall_rf = recall_score(y_test, y_pred, average='weighted', zero_division=0)

f1_rf = f1_score(y_test, y_pred, average='weighted', zero_division=0)

# Fix for class mismatch in classification_report

present_labels = np.unique(y_test)

present_class_names = label_encoder.inverse_transform(present_labels)

report = classification_report(

y_test,

y_pred,

labels=present_labels,

target_names=present_class_names,

output_dict=False,

zero_division=0

)

# Save fitted models to pickle

with open('model.pkl', 'wb') as f:

pickle.dump(rf_model, f)

with open('tfidf_vectorizer.pkl', 'wb') as f:

pickle.dump(tfidf, f)

with open('label_encoder.pkl', 'wb') as f:

pickle.dump(label_encoder, f)

# Plot 1: Top 10 Required Skills

skills_series = jobs_df['Required Skills'].dropna().str.replace('|', ',',
```

```
regex=False).str.cat(sep=',').split(',')
skills_series = pd.Series(skills_series).str.strip()
top_skills = skills_series.value_counts().head(10)
 plt.figure(figsize=(10, 6))
sns.barplot(x=top_skills.values, y=top_skills.index)
plt.title('Top 10 Required Skills')
bar_chart_skills = plot_to_base64()
# Plot 2: Top 20 Job Titles
job_title_counts = jobs_df['Job Title'].value_counts()
plt.figure(figsize=(12, 6))
job_title_counts.head(20).plot(kind='bar', color='skyblue')
plt.title('Top 20 Job Titles Count')
plt.xticks(rotation=45, ha='right')
bar_chart_titles = plot_to_base64()
# Plot 3: Location pie chart (Replacing Industry since it is replaced by Location in the Job
Profile)
top_n = 10
location_counts = jobs_df['Location'].value_counts().nlargest(top_n)
plt.figure(figsize=(8, 8))
plt.pie(location_counts, labels=location_counts.index, autopct='%1.1f%%', startangle=140,
colors=sns.color_palette("Set2"))
plt.title(f'Top {top_n} Job Location Distribution')
pie_chart = plot_to_base64()
return render(request, 'Users/training_form.html', {
'accuracy': round(accuracy * 100, 2),
'precision': round(precision_rf * 100, 2),
```

```
'recall': round(recall_rf * 100, 2),
'f1': round(f1_rf * 100, 2),
'report': report,
'bar_chart_skills': bar_chart_skills,
'bar_chart_titles': bar_chart_titles,
'pie_chart': pie_chart,
})
def check_and_load_models():
global tfidf, rf_model, label_encoder
if os.path.exists('model.pkl') and os.path.exists('tfidf_vectorizer.pkl') and
os.path.exists('label_encoder.pkl'):
try:
with open('model.pkl', 'rb') as f:
rf_model = pickle.load(f)
with open('tfidf_vectorizer.pkl', 'rb') as f:
tfidf = pickle.load(f)
with open('label_encoder.pkl', 'rb') as f:
label_encoder = pickle.load(f)
return
except Exception as e:
print("Pickle load failed, training dynamically:", e)
# Dynamic training fallback
jobs_df = pd.read_csv('media/jobs.csv', nrows=5000)
jobs_df['combined_features'] = jobs_df['Company'].fillna('') + " " + jobs_df['Required
Skills'].fillna('') + " " + jobs_df['Location'].fillna('') + " " + jobs_df['Salary'].fillna('') + " " +
jobs_df['Description'].fillna('')
```

```
tfidf.max_features = 500

X = tfidf.fit_transform(jobs_df['combined_features'])

y = label_encoder.fit_transform(jobs_df['Category'])

rf_model.n_estimators = 30

rf_model.max_depth = 12

rf_model.random_state = 42

rf_model.fit(X, y)

# Save fitted models to pickle

with open('model.pkl', 'wb') as f:

pickle.dump(rf_model, f)

with open('tfidf_vectorizer.pkl', 'wb') as f:

pickle.dump(tfidf, f)

with open('label_encoder.pkl', 'wb') as f:

pickle.dump(label_encoder, f)

def Prediction(request):

check_and_load_models()

user_profile = None

loginid = request.session.get('loginid')

if loginid:

try:

user_profile = UserRegistrationModel.objects.get(loginid=loginid)

except UserRegistrationModel.DoesNotExist:

pass

prediction_result = None

recommended_jobs = []

if request.method == 'POST':
```

```
skills = request.POST.get('skills', '')
education = request.POST.get('education', '')
try:
experience = int(request.POST.get('experience', '0'))
except ValueError:
experience = 0
preferences = request.POST.get('preferences', '')
previous_ratings = request.POST.get('previous_ratings', '')
# Update profile dynamically
if user_profile:
user_profile.skills = skills
user_profile.education = education
user_profile.experience = experience
user_profile.preferences = preferences
user_profile.previous_ratings = previous_ratings
user_profile.save()
# Build user profile search query string
user_profile_text = f"{skills} {education} {experience} years experience {preferences}
{previous_ratings}"
if user_profile_text.strip():
try:
# 1. Classification (predict job title)
user_vector = tfidf.transform([user_profile_text])
prediction = rf_model.predict(user_vector)
prediction_result = label_encoder.inverse_transform(prediction)[0]
# 2. Recommender (content-based top 5 matching jobs)
```

```
jobs_df = pd.read_csv('media/jobs.csv', nrows=5000)
jobs_df['combined_features'] = jobs_df['Company'].fillna('') + " " +
jobs_df['Required Skills'].fillna('') + " " + jobs_df['Location'].fillna('') + " " +
jobs_df['Salary'].fillna('') + " " + jobs_df['Description'].fillna('')
X_jobs = tfidf.transform(jobs_df['combined_features'])
similarities = cosine_similarity(user_vector, X_jobs).flatten()
top_indices = similarities.argsort()[::-1]
for idx in top_indices:
row = jobs_df.iloc[idx]
score = float(similarities[idx])
# Pre-split and clean skills string into a list
raw_skills = str(row['Required Skills']) if pd.notna(row['Required Skills']) else ''
skills_list = [s.strip() for s in raw_skills.replace('|', ',').split(',') if s.strip()][:8]
recommended_jobs.append({
'title': row['Job Title'],
'company': row['Company'] if 'Company' in row else 'N/A',
'skills': skills_list,
'location': row['Location'],
'salary': row['Salary'],
'description': row['Description'],
'apply_link': row['Apply Link'] if 'Apply Link' in row else '',
'score': round(score * 100, 2)
})
# Boost Hyderabad locations to ensure at least one is present (if matches exist)
india_jobs = [j for j in recommended_jobs if 'india' in j['location'].lower()]
worldwide_jobs = [j for j in recommended_jobs if 'india' not in j['location'].lower()]
```

```
india_jobs.sort(key=lambda x: -x['score'])

worldwide_jobs.sort(key=lambda x: -x['score'])

final_recommendations = []

hyderabad_jobs = [j for j in india_jobs if 'hyderabad' in j['location'].lower()]

other_india_jobs = [j for j in india_jobs if 'hyderabad' not in j['location'].lower()]

# Inject the top Hyderabad job first

if hyderabad_jobs:

final_recommendations.append(hyderabad_jobs[0])

# Fill other slots with remaining India jobs

for job in other_india_jobs:

if len(final_recommendations) < 5:

final_recommendations.append(job)

# Fill with additional Hyderabad jobs if slots left

if hyderabad_jobs:

for job in hyderabad_jobs[1:]:

if len(final_recommendations) < 5:

final_recommendations.append(job)

# Fill with Worldwide remote jobs

for job in worldwide_jobs:

if len(final_recommendations) < 5:

final_recommendations.append(job)

recommended_jobs = final_recommendations[:5]

except Exception as e:

prediction_result = f"Error: {str(e)}"

else:

prediction_result = "Please fill in skills and user profile details."
```

```
context = {

'prediction': prediction_result,

'recommended_jobs': recommended_jobs,

'user_profile': user_profile,

}

return render(request, 'users/predict_form.html', context)
```